

AUTOMATION TEST REPORT – BLAZEDEMO FLIGHT BOOKING PROJECT

Prepared By

Shivam Pal

Framework

Selenium WebDriver + Java + TestNG + Maven + Page Object Model (POM)

Project Type

Web Automation Testing

GitHub Repository-

https://github.com/Mrshivam09/Wipro_Capstone

1. Project Overview

The BlazeDemo Flight Booking Automation Project is developed to automate the complete flight booking process of the BlazeDemo web application. The primary objective of this project is to reduce manual testing effort, improve test execution speed, increase accuracy, and validate the end-to-end booking workflow through automation.

The framework is built using Selenium WebDriver with Java and follows the Page Object Model (POM) design pattern. The project automates important functionalities such as flight search, flight selection, passenger information entry, booking confirmation, screenshot capture, report generation, and result validation.

The framework also supports Data-Driven Testing using Excel files and generates detailed execution reports using TestNG and Extent Reports. Additionally, Continuous Integration and Continuous Deployment (CI/CD) practices have been implemented using Jenkins.

2. Objectives of the Project

The main objectives of this automation project are:

- Automate the complete flight booking workflow.
- Reduce repetitive manual testing activities.
- Improve test execution efficiency.
- Validate application functionality across multiple test data sets.
- Generate detailed execution reports.

- Integrate automation execution with Jenkins.
- Demonstrate industry-standard automation framework practices.

3. Technologies Used

The following technologies and tools were used during the development of this automation framework:

- Java
- Selenium WebDriver
- TestNG
- Maven
- Apache POI
- Page Object Model (POM)
- Git
- GitHub
- Jenkins
- Extent Reports
- Eclipse IDE

4. Framework Architecture

The automation framework follows the Page Object Model (POM) design pattern, which improves code reusability, maintainability, and scalability.

Project Structure

Base Package

Contains browser initialization and teardown methods.

Pages Package

Contains page classes representing application web pages.

- HomePage
- FlightSelectionPage
- PurchasePage
- ConfirmationPage

Tests Package

Contains all test execution classes.

- LaunchTest
- CrossBrowserTest

- SearchFlightTest
- FlightBookingTest

Utilities Package

Contains reusable utility classes.

- ConfigReader
- ExcelUtil
- ScreenshotUtil
- ExtentManager

Listeners Package

Contains TestNG listener implementation.

- TestListener

DataProviders Package

Signifies Data Driven approach.

- BookingDataProvider

Resources

Contains test data and configuration files.

- PassengerData.xlsx
- config.properties

5. Test Scenarios Automated

Scenario 1: Application Launch

Description

Verify that the BlazeDemo application launches successfully and the home page is displayed.

Steps

1. Launch the browser.
2. Navigate to the BlazeDemo URL.
3. Verify the page title.

Expected Result

The BlazeDemo home page should open successfully with the correct title.

Scenario 2: Search Flights

Description

Verify that users can search for available flights by selecting departure and destination cities.

Steps

1. Open the BlazeDemo application.
2. Select the departure city.
3. Select the destination city.
4. Click on **Find Flights**.

Expected Result

The list of available flights should be displayed successfully.

Scenario 3: Complete Flight Booking

Description

Verify that users can complete flight booking successfully.

Steps

1. Read passenger details from Excel.
2. Enter passenger information.
3. Enter payment details.
4. Click **Purchase Flight**.

Expected Result

The booking should be completed successfully.

Scenario 4: Validate Booking Confirmation

Description

Verify that the booking confirmation page is displayed after successful booking.

Steps

1. Complete the booking process.
2. Navigate to the confirmation page.
3. Validate the confirmation message.

Expected Result

The message "**Thank you for your purchase today!**" should be displayed successfully.

Scenario 5: Data-Driven Testing

Description

Verify the flight booking functionality using multiple test data records stored in an Excel file.

Steps

1. Read multiple rows from the Excel sheet.
2. Execute the booking flow for each record.
3. Validate successful booking.

Expected Result

All Excel data sets should execute successfully without errors.

Scenario 6: Cross Browser Testing

Description

Verify that the application works correctly on multiple browsers.

Expected Result

The application should function correctly and tests should pass on all supported browsers.

6. Reporting Mechanism

The framework generates multiple reports for execution tracking and analysis.

TestNG Report

Location:

test-output/index.html

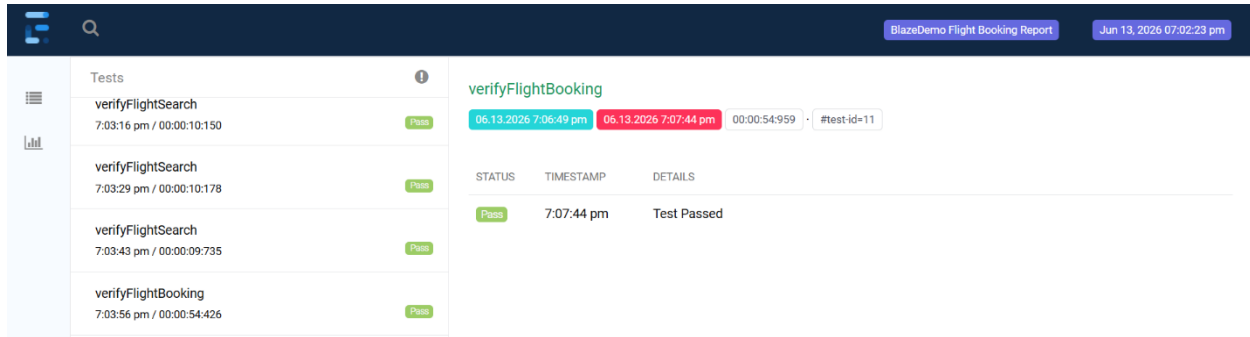
Features:

- Total Tests Executed
- Passed Tests
- Failed Tests
- Skipped Tests
- Execution Summary

Extent Report

Location:

Reports/ExtentReport.html



The screenshot displays a web-based Extent Report interface. At the top, there's a dark blue header with a search icon, a title 'BlazeDemo Flight Booking Report', and a timestamp 'Jun 13, 2026 07:02:23 pm'. The main content area is divided into two panels. The left panel, titled 'Tests', lists four test cases: 'verifyFlightSearch' (three instances) and 'verifyFlightBooking' (one instance), each with a 'Pass' status. The right panel, titled 'verifyFlightBooking', provides a detailed view of a specific test execution. It shows a timeline with two status changes: 'Pass' at 06:13:2026 7:06:49 pm and 'Fail' at 06:13:2026 7:07:44 pm, with a total duration of 00:00:54:959 and a test ID of #test-id=11. Below this, a table with columns 'STATUS', 'TIMESTAMP', and 'DETAILS' shows a single row: 'Pass' at '7:07:44 pm' with the detail 'Test Passed'.

Tests	
verifyFlightSearch	7:03:16 pm / 00:00:10:150 Pass
verifyFlightSearch	7:03:29 pm / 00:00:10:178 Pass
verifyFlightSearch	7:03:43 pm / 00:00:09:735 Pass
verifyFlightBooking	7:03:56 pm / 00:00:54:426 Pass

verifyFlightBooking			
06.13.2026 7:06:49 pm Pass 06.13.2026 7:07:44 pm Fail 00:00:54:959 #test-id=11			
STATUS	TIMESTAMP	DETAILS	
Pass	7:07:44 pm	Test Passed	

Features:

- Interactive Dashboard
- Pass/Fail Status
- Execution Timeline
- Detailed Logs
- Test Execution Summary
- Screenshot Integration

7. Screenshot Capture Functionality

The framework automatically captures screenshots during execution on failures and success .

Screenshots are captured for:

- Flight Search Page
- Flight Selection Page
- Purchase Page
- Confirmation Page
- Failed Test Cases

Benefits:

- Easier defect analysis
- Improved reporting
- Better execution evidence

8. Jenkins Integration

The automation framework is integrated with Jenkins Pipeline to enable continuous integration.

Jenkins Pipeline Stages

1. Checkout Source Code
2. Clean Project
3. Build Project
4. Execute TestNG Test Cases
5. Data-Driven Test Execution
6. Generate Extent Reports
7. Publish Reports
8. Archive Build Artifacts

The screenshot displays the Jenkins interface for a pipeline named 'BlazeDemo_Capstone' at build #5. The top section shows the pipeline script in a code editor, which is a declarative pipeline with stages for checkout, clean, compile, execute tests, archive reports, and post actions. The bottom section shows the pipeline execution graph, where all stages are marked as successful with green checkmarks. The stages are: Checkout SCM, Tool Install, Checkout Code, Clean Project, Compile Project, Execute TestNG Tests, Archive Reports, and Post Actions. The pipeline was started by Shivam Pal, queued 3 ms, and took 5 min 23 sec to complete. A 'Rerun' button is visible in the top right corner of the execution view.

```
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline Execution Completed
[Pipeline] echo
BlazeDemo Automation Build Successful
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Jenkins / BlazeDemo_Capstone / #5 / Stages

✓ #5

Started by Shivam Pal | Started 1 day 0 hr ago | Queued 3 ms | Took 5 min 23 sec | Artifacts

Start — End

Checkout SCM — Tool Install — Checkout Code — Clean Project — Compile Project — Execute TestNG Tests — Archive Reports — Post Actions

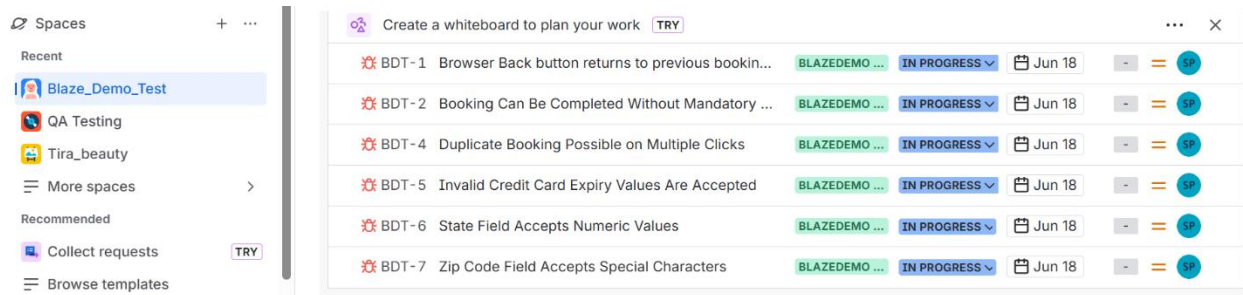
Rerun

Benefits

- Automated execution
- Scheduled builds
- Faster feedback on code changes
- CI/CD implementation
- Easy monitoring of test results

9. Defects Identified During Testing

Several defects were identified during execution:

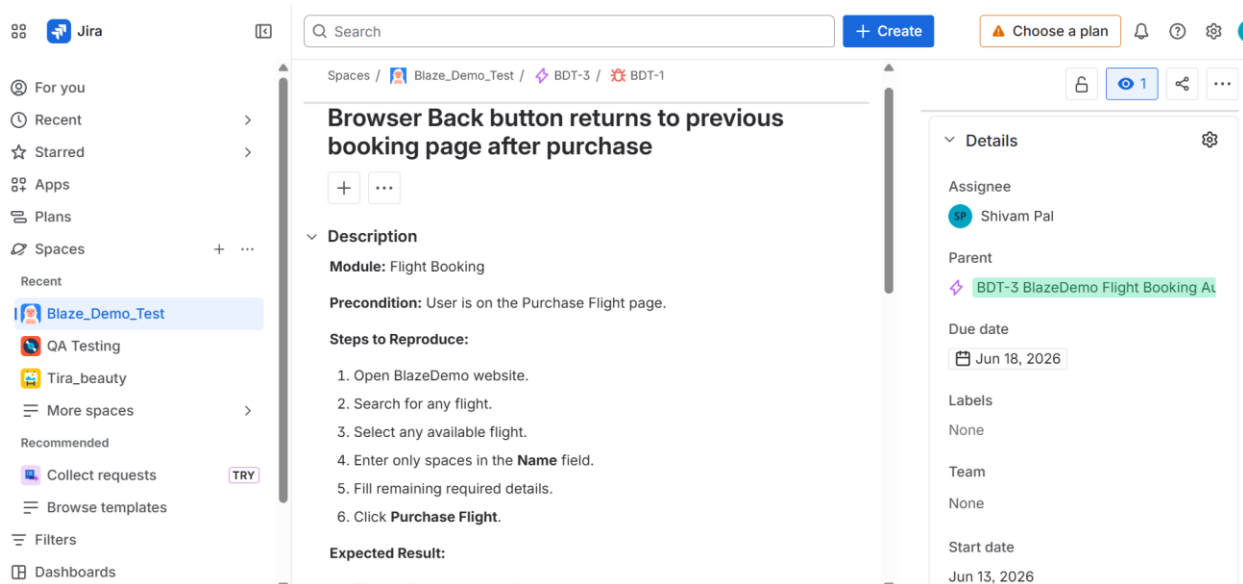


The screenshot shows the Jira interface with a list of defects. The left sidebar shows the 'Spaces' section with 'Blaze_Demo_Test' selected. The main area displays a table of defects:

Defect ID	Defect Description	Project	Status	Due Date	Priority
BDT-1	Browser Back button returns to previous bookin...	BLAZEDEMO ...	IN PROGRESS	Jun 18	SP
BDT-2	Booking Can Be Completed Without Mandatory ...	BLAZEDEMO ...	IN PROGRESS	Jun 18	SP
BDT-4	Duplicate Booking Possible on Multiple Clicks	BLAZEDEMO ...	IN PROGRESS	Jun 18	SP
BDT-5	Invalid Credit Card Expiry Values Are Accepted	BLAZEDEMO ...	IN PROGRESS	Jun 18	SP
BDT-6	State Field Accepts Numeric Values	BLAZEDEMO ...	IN PROGRESS	Jun 18	SP
BDT-7	Zip Code Field Accepts Special Characters	BLAZEDEMO ...	IN PROGRESS	Jun 18	SP

Defect 1

Browser Back Button Returns to Previous Booking Page.



The screenshot shows the Jira interface with the details of Defect 1. The left sidebar shows the 'Spaces' section with 'Blaze_Demo_Test' selected. The main area displays the details of the defect:

Browser Back button returns to previous booking page after purchase

Description

Module: Flight Booking

Precondition: User is on the Purchase Flight page.

Steps to Reproduce:

1. Open BlazeDemo website.
2. Search for any flight.
3. Select any available flight.
4. Enter only spaces in the **Name** field.
5. Fill remaining required details.
6. Click **Purchase Flight**.

Expected Result:

- The application should display a validation message and should not allow

Details

Assignee: Shivam Pal

Parent: BDT-3 BlazeDemo Flight Booking At

Due date: Jun 18, 2026

Labels: None

Team: None

Start date: Jun 13, 2026

Defect 2

Booking Can Be Completed Without Mandatory Details.

This screenshot shows a Jira issue in the 'Blaze_Demo_Test' space. The issue title is 'Booking Can Be Completed Without Mandatory Details'. The description includes the following details:

- Module:** Flight Booking
- Precondition:** User is on the Purchase Flight page.
- Steps to Reproduce:**
 1. Open BlazeDemo.
 2. Search and select any flight.
 3. Leave mandatory fields empty.
 4. Click **Purchase Flight**.
- Expected Result:**
 - Validation messages should be displayed and booking should not proceed.
- Actual Result:**

The right sidebar shows the issue details, including the assignee 'Shivam Pal', parent issue 'BDT-3 BlazeDemo Flight Booking At', due date 'Jun 18, 2026', and start date 'Jun 13, 2026'.

Defect 3

Duplicate Booking Possible on Multiple Clicks.

This screenshot shows a Jira issue in the 'Blaze_Demo_Test' space. The issue title is 'Duplicate Booking Possible on Multiple Clicks'. The description includes the following details:

- Module:** Payment
- Precondition:** User is on the Purchase Flight page.
- Steps to Reproduce:**
 1. Fill all booking details.
 2. Double-click the **Purchase Flight** button quickly.
- Expected Result:**
 - Only one booking should be processed.
- Actual Result:**
 - Multiple booking requests may be submitted.
- Severity:** High
- Priority:** Medium

The right sidebar shows the issue details, including the assignee 'Shivam Pal', parent issue 'BDT-3 BlazeDemo Flight Booking At', due date 'Jun 18, 2026', and start date 'Jun 13, 2026'.

Defect 4

Invalid Credit Card Expiry Values Are Accepted.

This screenshot shows a Jira issue in the 'Blaze_Demo_Test' space, specifically under the 'BDT-3' project. The issue title is 'Invalid Credit Card Expiry Values Are Accepted'. The description includes a module of 'Payment', a precondition that the user is on the 'Purchase Flight' page, and five steps to reproduce the defect: searching for a flight, entering '99' for the month, '0000' for the year, and clicking 'Purchase Flight'. The expected result is that the application should reject invalid expiry values and show an error message. The right-hand sidebar shows details such as the assignee 'Shivam Pal', the parent issue 'BDT-3 BlazeDemo Flight Booking At', a due date of 'Jun 18, 2026', and a start date of 'Jun 13, 2026'.

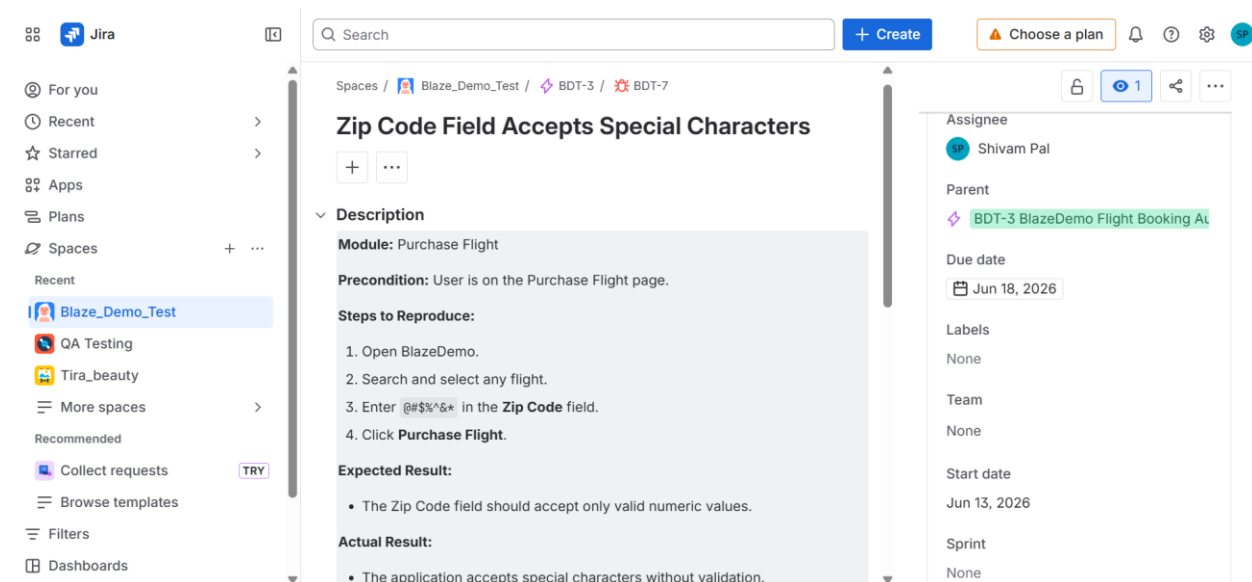
Defect 5

State Field Accepts Numeric Values.

This screenshot shows a Jira issue in the 'Blaze_Demo_Test' space, specifically under the 'BDT-6' project. The issue title is 'State Field Accepts Numeric Values'. The description includes a module of 'Purchase Flight', a precondition that the user is on the 'Purchase Flight' page, and five steps to reproduce the defect: opening 'BlazeDemo', searching for a flight, entering '123456' in the 'State' field, filling in the remaining details, and clicking 'Purchase Flight'. The expected result is that the 'State' field should accept only valid alphabetic characters. The right-hand sidebar shows details such as the assignee 'Shivam Pal', the parent issue 'BDT-3 BlazeDemo Flight Booking At', a due date of 'Jun 18, 2026', and a start date of 'Jun 13, 2026'.

Defect 6

Zip Code Field Accepts Special Characters.



All defects were documented and tracked through Jira.

10. Challenges Faced and Solutions

Challenge 1

Cross-browser execution and browser driver compatibility issues.

Solution

Implemented WebDriverManager and browser-specific configuration using TestNG parameters for Chrome and Firefox execution.

Challenge 2

Reading multiple test data records from Excel.

Solution

Implemented Apache POI and TestNG Data Providers to support Data-Driven Testing with multiple datasets.

Challenge 3

Detailed reporting and execution evidence requirement.

Solution

Integrated Extent Reports with TestNG Listeners and automatic screenshot capture for test execution.

11. Project Achievements

- Developed Selenium WebDriver Automation Framework
- Implemented Page Object Model (POM)
- Implemented Data-Driven Testing using Excel
- Integrated TestNG Framework
- Implemented Cross-Browser Testing (Chrome & Firefox)
- Generated Extent Reports for test execution
- Implemented Automatic Screenshot Capture
- Externalized configuration using config.properties
- Integrated Jenkins for Continuous Integration (CI)
- Managed source code using Git and GitHub

12. Conclusion

The BlazeDemo Flight Booking Automation Framework successfully automates the complete flight booking process using Selenium WebDriver, Java, and TestNG. The framework follows the Page Object Model (POM) architecture to ensure code reusability and maintainability. It supports Data-Driven Testing using Excel and provides detailed execution reports through Extent Reports and TestNG. Cross-browser testing, screenshot capture, and Jenkins integration enhance the reliability and efficiency of the framework. Overall, the project demonstrates industry-standard automation testing practices and provides a scalable and maintainable solution for web application testing.